Testing Lab: File Sharing & Collaboration

Overview

This testing lab covers file sharing links, shared folders, upload requests, and collaborative features including member management and access control.

Test Environment Setup

Prerequisites

- FileFlow running with multiple user accounts
- Test users created:
  - Admin: `admin` / `admin123`
  - User 1: `user1` / `password123`
  - User 2: `user2` / `password123`
  - User 3: `user3` / `password123`
- Test files and folders prepared
- Access to email for verifying share notifications (optional)

Feature 1: Basic File Sharing

Test 1.1: Create Share Link

Objective: Verify creating basic public share links
Steps: 1. Login as `user1` 2. Right-click on file: `report.pdf` 3. Select “Share” 4. Configure: No password, no expiration 5. Click “Create Share Link” 6. Copy generated link

Expected Results: - ✓ Share link generated (format: `/s/[token]`) - ✓ Link copied to clipboard - ✓ Share created successfully - ✓ File accessible via link - ✓ Can preview file without login

---

Test 1.2: Share Link with Password

Objective: Verify password-protected shares
Steps: 1. Right-click on file 2. Select “Share” 3. Enable password: `SecurePass123!` 4. Create share link 5. Copy link 6. Logout 7. Open share link in new tab 8. Enter password

Expected Results: - ✓ Share link created - ✓ Password prompt appears when accessing - ✓ Correct password grants access - ✓ Incorrect password rejected (password page shown again with an error) - ✓ File accessible only after password - ✓ File content hidden from non-authenticators - ✓ See Test 6.4 for the attempt limit (HTTP 429 after 5 failures)

Test 1.3: Share Link with Expiration

Objective: Verify time-limited share links
Steps: 1. Create share link 2. Set expiration: 24 hours 3. Verify time shown 4. Access link now (should work) 5. Wait or set clock forward 25 hours 6. Try to access link again

Expected Results: - ✓ Link works immediately - ✓ Expiration time clearly shown in share settings - ✓ Link becomes inaccessible after expiration - ✓ Expired link returns HTTP 404 with the same message as an unknown link (“Share link not found or expired”) - ✓ Share can be extended if needed

---

Test 1.4: Revoke Share Link

Objective: Verify disabling existing shares
Steps: 1. Create share link 2. Share with someone 3. Navigate to Settings > Shares 4. Find share for `report.pdf` 5. Click “Revoke” 6. Confirm revocation 7. Try to access previously shared link

Expected Results: - ✓ Share revoked immediately - ✓ Link no longer works - ✓ HTTP 404: “Share link not found or expired” - ✓ No longer visible in share list - ✓ Can create new share if needed

---

Test 1.5: Download via Share Link

Objective: Verify downloading files via share
Steps: 1. Create share link for file 2. Copy link 3. Open in incognito window 4. Click “Download” button

Expected Results: - ✓ Download button visible on share page - ✓ File downloads without login - ✓ File content matches original - ✓ Filename preserved - ✓ File size correct

---

Test 1.6: Share Multiple Files

Objective: Verify sharing folders/multiple files
Steps: 1. Right-click on folder with 5 files 2. Select “Share” 3. Create share link 4. Access share link

Expected Results: - ✓ Share link created for folder - ✓ Link shows folder contents - ✓ All 5 files visible and downloadable - ✓ Folder structure preserved - ✓ Can download entire folder as ZIP

Test 1.7: Download-Limited Link

Objective: Verify a link with a download cap cannot be downloaded more times than allowed
Steps: 1. Create a share link with “Limit downloads” enabled and the cap set to 1 2. Download the file once via the link 3. Try to download it a second time 4. Create another one-time link and send 20 parallel requests to it (e.g. a `curl` loop with &)

Expected Results: - ✓ First download succeeds; the share list shows `1 / 1` - ✓ Second download returns HTTP 404 (“reached its download limit”) - ✓ With parallel requests, exactly one request receives the file; all others get 404 - ✓ If a download fails on the server side, it is not counted

---

Test 1.8: Range Requests on Download-Limited Links

Objective: Verify partial downloads cannot bypass the download cap
Steps: 1. Create a share link with a cap of 2 downloads 2. Request it with `curl -H "Range: bytes=0-99" <link>` three times 3. Create a share link without a cap and request it with the same Range header

Expected Results: - ✓ Capped link: the Range header is ignored; each request returns HTTP 200 with the whole file and counts as one download - ✓ Capped link: the third request returns HTTP 404 (cap reached) - ✓ Uncapped link: HTTP 206 with only the requested bytes (`Content-Range` header present)

Test 1.9: Share Link Follows Rename and Move, Dies on Delete

Objective: Verify public links stay attached to the shared resource
Steps: 1. Create a share link for `report.pdf` 2. Rename it to `report-final.pdf`, open the link 3. Move it to another folder, open the link 4. Delete the file, open the link

Expected Results: - ✓ After rename and move, the link still downloads the file (shown under its new name) - ✓ After delete, the link returns HTTP 404 and disappears from the share list - ✓ Restoring the file from Trash does not bring the link back

---

Test 1.10: Creator Loses Access

Objective: Verify a link stops working when its creator can no longer read the file
Steps: 1. As `user1`, create a share link for a file 2. As admin, disable `user1` (or remove the read permission / restrict their group's allowed paths so the file is excluded) 3. Open the link 4. Re-enable `user1` (restore the access) and open the link again

Expected Results: - ✓ Step 3: HTTP 404 with the same message as an unknown or expired link (the reason is not revealed) - ✓ Step 4: the link works again (it was not deleted, only refused while the creator lacked access)

Feature 2: Shared Folders

Test 2.1: Create Shared Folder

Objective: Verify creating collaborative folder
Steps: 1. Login as `user1` 2. Right-click in `/files/` 3. Select "Create Shared Folder" 4. Name: `Project_2024_Q2` 5. Set members: - `user2` (Read-only) - `user3` (Read-Write) 6. Click "Create"

Expected Results: - ✓ Shared folder created at `/files/Project_2024_Q2/` - ✓ Folder visible only to members + creator - ✓ `user1` has full control (owner) - ✓ `user2` has read-only access - ✓ `user3` can read and write - ✓ Other users cannot access - ✓ Creating the shared folder requires `user1`'s **own** read access to the path (a path only reachable through a share received from someone else is refused with 403) - ✓ Granting Read-Write to a member requires `user1`'s own write access to the path (refused with 403 otherwise)

Test 2.2: Add Member to Shared Folder

Objective: Verify adding members after creation
Steps: 1. Open shared folder settings 2. Click "Add Member" 3. Select `user2` 4. Set permission: "Read-Write" 5. Click "Add"

Expected Results: - ✓ Member added successfully - ✓ `user2` can now access folder - ✓ Permission applied correctly - ✓ Member list updated - ✓ Notification sent to member (optional)

Test 2.3: Change Member Permissions

Objective: Verify updating member permissions
Steps: 1. Open shared folder settings 2. Find `user3` in members list 3. Change permission: "Read-Write" → "Read-Only" 4. Confirm change 5. Login as `user3` 6. Try to upload file to shared folder

Expected Results: - ✓ Permission changed immediately - ✓ `user3` can no longer upload - ✓ Error message: "You don't have permission to upload here" - ✓ Can still download and read files - ✓ Changes apply in real-time

---

Test 2.4: Remove Member from Shared Folder

Objective: Verify removing members
Steps: 1. Open shared folder settings 2. Find member in list 3. Click "Remove" next to member 4. Confirm removal 5. Login as removed member 6. Try to access shared folder

Expected Results: - ✓ Member removed successfully - ✓ Member can no longer access folder - ✓ Error message: "Access denied" - ✓ Folder no longer appears in member's list - ✓ Removal is immediate

---

Test 2.5: Shared Folder Inheritance

Objective: Verify permissions inherited to new files
Steps: 1. Upload file to read-only shared folder as owner 2. Login as read-only member 3. Try to download uploaded file

Expected Results: - ✓ All files in shared folder inherit folder permissions - ✓ Read-only members can download - ✓ Read-only members cannot modify - ✓ Read-write members can modify - ✓ Permissions consistent across all files

---

Test 2.6: Members Never Exceed the Creator's Rights

Objective: Verify shared-folder access is bounded by the creator's current rights
Steps: 1. `user1` (editor) creates a shared folder and gives `user3` Read-Write 2. As admin, change `user1` to a read-only role (or remove their write permission) 3. As `user3`, try to upload to the shared folder; then try to read a file 4. As admin, disable `user1` or restrict their allowed paths to exclude the folder 5. As `user3`, try to open the shared folder

Expected Results: - ✓ Step 3: upload refused (403); read still allowed - ✓ Step 5: access refused; members lose access as soon as the creator does - ✓ Restoring `user1`'s rights restores the members' access without changing the membership

---

Feature 3: Upload Requests

Test 3.1: Create Upload Request

Objective: Verify creating public upload links
Steps: 1. Login as `user1` 2. Right-click on folder 3. Select "Create Upload Request" 4. Set: - Folder destination: `/files/uploads/` - Max file size: 100MB - Allowed file types: images, documents - Expiration: 7 days 5. Click "Create" 6. Copy link

Expected Results: - ✓ Upload request created (only if `user1` has their **own** write access to the folder) - ✓ Public link generated - ✓ Link accessible without login - ✓ Upload page shows constraints - ✓ File restrictions displayed

---

Test 3.2: Upload via Upload Request

Objective: Verify anonymous file uploads
Steps: 1. Copy upload request link 2. Open in new incognito window 3. Drag or select file to upload 4. Submit file 5. Verify upload completion

Expected Results: - ✓ Upload form shows file size limit - ✓ Drag-and-drop works - ✓ Progress shown during upload - ✓ File uploaded to specified folder - ✓ File available to folder owner - ✓ Uploader info logged (IP, timestamp) - ✓ Uploading a file whose name already exists never overwrites it: the new file is saved as `name (1).ext`, `name (2).ext`, etc.

Test 3.3: Upload Request File Restrictions

Objective: Verify file type restrictions
Steps: 1. Create upload request: allowed types = images only 2. Try to upload: - Image file (.jpg) → should succeed - Document file (.pdf) → should fail - Executable (.exe) → should fail

Expected Results: - ✓ Image uploads successfully - ✓ Document rejected: "File type not allowed" - ✓ Executable rejected: "File type not allowed" - ✓ Clear error messages - ✓ Restrictions enforced

---

Test 3.4: Upload Request Size Limit

Objective: Verify the server-side cap on anonymous uploads
Steps: 1. Start the server with `PUBLIC_UPLOAD_MAX_MB=50` (default 100, never above `MAX_UPLOAD_MB`, default 200) 2. Create an upload request 3. Try to upload: - 30MB file → should succeed - 60MB file → should fail

Expected Results: - ✓ 30MB file uploads - ✓ 60MB file rejected with HTTP 400 ("Upload exceeds the maximum allowed size") - ✓ No partial or temporary file left in the target folder or in `.file-flow/uploads` - ✓ The limit applies to the total of one submission, independently of the authenticated upload limit

---

Test 3.5: Revoke Upload Request

Objective: Verify disabling upload links
Steps: 1. Create upload request 2. Share link with someone 3. Navigate to Settings > Upload Requests 4. Find request 5. Click "Revoke" 6. Try to access link again

Expected Results: - ✓ Upload request disabled - ✓ Link no longer works - ✓ Message: "Upload request not found" - ✓ Files already uploaded remain

---

Test 3.6: Read-Only User Cannot Create an Upload Request

Objective: Verify an upload request cannot grant more than the creator's own write access
Steps: 1. Login as a Viewer (read-only) who can browse `/files/shared/` 2. Try to create an upload request on `/files/shared/` (UI or `POST /api/upload-requests`) 3. Login as `user2`, who only has **read or write access through an internal share** on `/files/user1/proj/`, and try the same on that folder

Expected Results: - ✓ Both requests refused with HTTP 403; no link created - ✓ Only users with their own write access to the folder can create upload requests

---

Test 3.7: Password Before Files on Protected Upload Requests

Objective: Verify files are not accepted before the password is checked
Steps: 1. Create a password-protected upload request 2. Submit the form normally with the correct password and a file 3. Send a multipart request with the `files` field **before** the `pwd` field (e.g. `curl -F files=@a.txt -F pwd=secret <link>`) 4. Submit with a wrong password 5 times, then once more with the correct password

Expected Results: - ✓ Step 2: upload succeeds (the form sends the password first) - ✓ Step 3: refused with "Please enter the password before uploading."; nothing written to disk - ✓ Step 4: each wrong password shows "Incorrect password"; the 6th request returns HTTP 429 for 15 minutes

---

Test 3.8: Upload Request Lifecycle and Creator Access

Objective: Verify upload requests follow their folder and stop when the creator loses write access
Steps: 1. Create an upload request on `/files/inbox/` 2. Rename the folder to `/files/inbox-2026/` and upload via the link 3. As admin, remove the creator's write permission; upload via the link again 4. Restore the permission, then delete the folder and open the link

Expected Results: - ✓ Step 2: the file lands in `/files/inbox-2026/` - ✓ Step 3: HTTP 404 ("Upload request not found or expired") - ✓ Step 4: the link returns 404 and the request is removed from the list

---

Feature 4: Access Control & Permissions

Test 4.1: Group-Based Access

Objective: Verify folder access by group membership
Steps: 1. Create group: "Marketing Team" with `user2`, `user3` 2. Create shared folder: "Marketing Assets" 3. Grant access to group (not individuals) 4. Login as `user2` (in group) 5. Verify access to folder

Expected Results: - ✓ Group created with members - ✓ Folder shared with group - ✓ `user2` can access folder - ✓ `user3` can access folder - ✓ Others cannot access

---

Test 4.2: Path-Based Restrictions

Objective: Verify members can only access specific paths
Steps: 1. Give `user2` access only to `/files/public/` 2. Login as `user2` 3. Try to access `/files/public/` → should work 4. Try to access `/files/private/` → should fail

Expected Results: - ✓ Public folder accessible - ✓ Private folder shows "Access denied" - ✓ Cannot navigate outside permitted path - ✓ Cannot see restricted directories in listing

---

Test 4.3: Owner Cannot Be Removed

Objective: Verify folder owner cannot lose ownership
Steps: 1. Create shared folder as `user1` 2. Try to remove `user1` from members 3. Try to change `user1`'s permission

Expected Results: - ✓ Remove button disabled for owner - ✓ Cannot change owner permission - ✓ Owner has full control always - ✓ Clear indication "You are the owner"

Collaboration Tests

Test 5.1: Concurrent Edits Notification

Objective: Verify awareness of concurrent changes
Steps: 1. `user1` creates shared folder with `user2` 2. Both upload files to folder simultaneously 3. Both check folder contents

Expected Results: - ✓ Both uploads complete successfully - ✓ Both see all files (after refresh) - ✓ No file corruption - ✓ Both uploads recorded in audit log

---

Test 5.2: Activity Log in Shared Folder

Objective: Verify sharing activity is tracked
Steps: 1. Navigate to shared folder 2. Click “Activity” or “History” 3. Review recent actions

Expected Results: - ✓ Activity log shows all actions - ✓ Shows who did what and when - ✓ Upload actions show username and timestamp - ✓ Delete actions show restoring info - ✓ Member changes logged

---

Security Tests

Test 6.1: Share Token Expiration

Objective: Verify share links cannot be reused after revocation
Steps: 1. Create share link 2. Save the link/token 3. Revoke share 4. Try to access share with saved token

Expected Results: - ✓ Revocation invalidates all tokens - ✓ Old token doesn’t work - ✓ Cannot access files with old token - ✓ No fallback or workaround

---

Test 6.2: Share Password Security

Objective: Verify passwords not exposed in logs
Steps: 1. Create share with password: `MySecure!Pass123` 2. Check application logs 3. Check browser history

Expected Results: - ✓ Password not visible in logs - ✓ Password not in browser history - ✓ No plaintext password stored - ✓ Only hash/encrypted form stored

---

Test 6.3: Cross-Site Request Forgery (CSRF) Protection

Objective: Verify sharing operations protected against CSRF
Steps: 1. Create share link 2. Open another website in different tab 3. Try to trigger share revocation from that site 4. Verify nothing happens

Expected Results: - ✓ CSRF tokens validated - ✓ Cross-site requests rejected - ✓ Requires valid CSRF token - ✓ Requests from FileFlow work, external sites blocked

---

Test 6.4: Share Link Password Throttling

Objective: Verify brute force on link passwords is limited
Steps: 1. Create a password-protected share link 2. Submit 5 wrong passwords 3. Submit the correct password 4. Restart the server (lockouts are kept in memory), then from one IP submit 5 wrong passwords spread over 3 **different** protected links 5. Open any other protected link from that IP and submit its correct password 6. Wait 15 minutes (or restart the server) and submit the correct password on the first link

Expected Results: - ✓ Step 2: each wrong password shows the password page with an error - ✓ Step 3: HTTP 429 ("Too many attempts. Try again later."), even with the correct password: the link is locked for 15 minutes - ✓ Step 5: HTTP 429: failures are also counted per IP, and after 5 the IP is locked on every protected link - ✓ Step 6: the correct password works again - ✓ The same limit applies to password-protected upload requests (`/r/{token}`)

Note: The per-link lock also blocks legitimate recipients for 15 minutes (known open point N1).

Test Results Summary

Test ID	Test Name	Status	Notes
1.1	Create Share	☐	
1.2	Share with Password	☐	
1.3	Share Expiration	☐	
1.4	Revoke Share	☐	
1.5	Download via Share	☐	
1.6	Share Multiple Files	☐	
1.7	Download-Limited Link	☐	
1.8	Range on Limited Link	☐	
1.9	Link Follows Rename/Delete	☐	
1.10	Creator Loses Access	☐	
2.1	Create Shared Folder	☐	
2.2	Add Member	☐	
2.3	Change Permissions	☐	
2.4	Remove Member	☐	
2.5	Permission Inheritance	☐	
2.6	Bounded by Creator Rights	☐	
3.1	Create Upload Request	☐	
3.2	Upload via Request	☐	
3.3	File Restrictions	☐	
3.4	Size Limit	☐	
3.5	Revoke Upload Request	☐	
3.6	Read-Only Cannot Create Request	☐	
3.7	Password Before Files	☐	
3.8	Request Lifecycle	☐	
4.1	Group-Based Access	☐	
4.2	Path Restrictions	☐	
4.3	Owner Protection	☐	
5.1	Concurrent Edits	☐	
5.2	Activity Log	☐	
6.1	Token Expiration	☐	
6.2	Password Security	☐	
6.3	CSRF Protection	☐	
6.4	Link Password Throttling	☐	

Sign-Off

- **Tester Name**: _____
- **Test Date**: _____
- **Total Tests**: 33
- **Passed**: ____
- **Failed**: ____
- **Blocked**: ____

Conclusion: ☐ Ready for Production | ☐ Needs Fixes | ☐ Blocked

---

Version History

Date	Version	Changes	Author
2024-05-05	1.0	Initial lab	Driss OUAAZIZ
2026-10-27	1.1	Uniform 404 for expired links; shared-folder and upload-request creation require the creator’s own rights; no overwrite on public uploads; `PUBLIC_UPLOAD_MAX_MB`; tests 1.7–1.10, 2.6, 3.6–3.8 and 6.4 added (download cap, Range, link lifecycle, creator access, password-before-files, 429 throttling)	Driss OUAAZIZ